

OPERATING SYSTEMS — COURSE PROJECT REPORT

AKIL

Adaptive Kernel Intelligence Layer for Runtime-Aware Container Scheduling

Literature Review · Problem Statement · Project Objectives · Novelty/Innovation

Prepared: August 2026

1. Literature Review

Kubernetes scheduling and OS-aware resource management have both been studied extensively in industry and academic systems research. This project builds on, rather than departs from, that body of work. The review below groups relevant prior work into four categories and, for each, states specifically what it does and does not cover — since that gap is what defines the project's contribution.

1.1 Production cluster schedulers

Google's Borg and its successor Autopilot are the most cited references for large-scale cluster scheduling. Both incorporate historical resource-usage prediction and reactive resource adjustment at the scale of hundreds of thousands of machines. However, published descriptions of these systems center on resource-usage prediction (CPU/memory sizing over time) rather than fine-grained kernel runtime behavior such as cache locality, page fault rate, or lock contention at scheduling time. This project's telemetry layer targets exactly this uncovered signal class.

1.2 Alternative scheduling architectures

The Firmament scheduler (University of Cambridge) reformulates cluster scheduling as a min-cost flow optimization problem and has been benchmarked directly against Kubernetes' default scheduler, showing scheduling-quality improvements at scale. Firmament's contribution is in the optimization formulation itself, not in the acquisition of new runtime signal types — it still largely operates over resource-based inputs. This project is complementary: its telemetry layer could, in principle, feed a Firmament-style optimizer as well as the simpler Score-plugin approach used here.

1.3 Kubernetes-native topology and NUMA awareness

Kubernetes SIG-Scheduling already ships a Topology Manager and NUMA-aware resource allocation for select resource types (notably CPU pinning and device plugins), and exposes some topology hints to the scheduler. This existing work substantially narrows what can be claimed as novel around NUMA-awareness specifically — this project cannot claim to introduce NUMA-aware placement in general. What it can credibly investigate is combining topology hints already available from Kubernetes with dynamically observed runtime signals (cache locality, lock contention) that are not part of the existing Topology Manager's input space.

1.4 eBPF-based observability tooling

Cilium, Pixie, and Parca are widely deployed eBPF-based observability projects that already extract low-level kernel signals — network flows, function-level profiling, memory allocation patterns — at production scale with demonstrated low overhead. These projects establish that the telemetry-collection half of this proposal (Section on Architecture in the companion documentation) is technically well-trodden and low-risk to implement. None of them, however, feed their telemetry back into a live scheduling decision; they are observability tools, not

scheduling inputs. This is the specific gap the project's aggregation-to-scheduler pipeline addresses.

1.5 Summary of the literature gap

Prior work	Contribution	What it does not cover
Google Borg / Autopilot	Large-scale resource-usage prediction and reactive adjustment	Kernel-level runtime behavior (cache, lock contention, page faults) as a scheduling input
Firmament	Flow-network optimization for scheduling quality	New runtime telemetry signal acquisition
Kubernetes Topology Manager	NUMA/device-level topology hints for CPU pinning	Dynamic, continuously observed runtime signals (contention, cache misses)
Cilium / Pixie / Parca	Low-overhead eBPF telemetry at production scale	Closing the loop into a live scheduler decision

Table 1. Positioning of AKIL relative to existing systems and tooling.

A formal, citation-backed literature review (with full academic references, e.g. from OSDI/NSDI/EuroSys proceedings) is a required next step before any claims of novelty are finalized; this section should be treated as a scoping review to guide implementation, not a substitute for that later, more rigorous search.

2. Problem Statement

Kubernetes' default scheduler (kube-scheduler) makes placement decisions using static, coarse-grained infrastructure metrics: requested and available CPU, memory, node labels, taints/tolerations, and declared affinity rules. These metrics say nothing about what actually happens inside the operating system kernel once a workload is running — page fault frequency, cache locality, NUMA memory access latency, lock and mutex contention, and I/O wait behavior are all invisible to the scheduler.

This creates a systematic blind spot: two pods with identical resource requests can have entirely different runtime behavior, yet the scheduler treats them identically. One pod may be cache-friendly and CPU-bound; another may generate heavy page-fault activity or contend on locks with a co-located process. The result is avoidable performance degradation — increased memory access latency from poor NUMA placement, cache thrashing from destructive co-location, and reactive rather than proactive handling of failure (Kubernetes generally responds only after a problem manifests as an OOM kill or a failed liveness probe, not before).

The problem, stated precisely: container orchestration scheduling decisions are made with strictly less information than the operating system already possesses about workload behavior, and no standard mechanism currently closes that information gap at scheduling time.

3. Project Objectives

3.1 General objective

To design and implement a working, kernel-telemetry-informed scheduling extension for Kubernetes, and to evaluate whether incorporating OS-level runtime signals into scheduling decisions measurably improves placement quality relative to the default scheduler.

3.2 Specific objectives

1. Implement a low-overhead, eBPF-based kernel telemetry collector capable of sampling page fault rate, cache miss rate, lock/mutex contention, and context-switch frequency on a per-workload basis.
2. Build an aggregation layer that converts raw kernel events into a stable, low-cardinality runtime profile queryable within a scheduler's latency budget.
3. Implement a Kubernetes scheduler plugin, using the standard Kubernetes Scheduling Framework, that scores candidate nodes using the aggregated runtime profile in addition to standard resource-based criteria.
4. Design and run a synthetic, deliberately memory- and cache-sensitive benchmark workload capable of exposing a measurable difference between default and telemetry-aware placement.
5. Quantitatively evaluate the system: compare latency, cache-miss rate, and migration/eviction counts between default kube-scheduler placement and AKIL-informed placement on identical workloads and cluster conditions.
6. Document the system's architecture, evaluation results, and a scoped novelty claim in a form suitable both as a course deliverable and as the starting point for a future prior-art and patentability assessment.

3.3 Explicitly out of scope for this phase

- Predictive migration based on trend forecasting.
- Reinforcement-learning-based policy refinement across deployments.
- Energy/thermal-aware scheduling and behavioral-fingerprint-based security quarantine (documented as future extensions only).

4. Novelty / Innovation

Per the literature review in Section 1, neither kernel-level telemetry collection nor NUMA/topology-aware scheduling is independently novel — both exist in mature, published form. The specific, narrower claim this project makes is about the combination and the closed-loop architecture connecting them:

7. Continuous, low-overhead kernel-level behavioral telemetry — not just resource usage — collected via eBPF, covering signal types (lock contention, cache locality) not currently exposed to Kubernetes' scheduler.
8. A defined aggregation step that converts this telemetry into a per-workload runtime profile cheap enough to query synchronously inside a live scheduling decision, rather than only for offline observability (the gap identified against Cilium/Pixie/Parca in Section 1.4).
9. A scheduler plugin, built on the standard, reusable Kubernetes Scheduling Framework, that consumes this profile at Score time — closing the loop from kernel behavior to placement decision within the workload's own runtime, rather than only at initial deployment.

This claim is intentionally narrow and should be treated as a hypothesis to validate — and likely narrow further — once the formal literature/prior-art search (Section 1.5) is complete. Broad framing ("AI-powered", "self-evolving", "predictive everything") is deliberately avoided here: patent claims and academic novelty claims alike are strongest when they describe one specific, testable mechanism and its measurable effect, not an ambitious system description. The evaluation results from Objective 5 (Section 3.2) are what will ultimately determine whether this claim holds up, and are a prerequisite before pursuing either a patent filing or a publication claim of novelty.
